

Universidade Estadual de Londrina

Sistemas Distribuídos

Shared Distributed Write Board (SDWB)

Quadro branco distribuído com coordenador migrante

Marco Antônio Cottorello Henry

Londrina

15 de junho de 2026

Sumário

1	Introdução	2
2	Arquitetura	3
2.1	Coordenador in-process e migração	3
2.2	Modelo de dados	3
2.3	Organização do código	3
3	Protocolo de mensagens (gRPC)	4
3.1	Serviço <code>ServicoNomes</code>	4
3.2	Serviço <code>Coordenador</code>	4
3.3	Serviço <code>No</code> (eleição)	5
4	Descoberta e <i>onboarding</i>	5
5	Ordenação e consistência	5
6	Exclusão mútua	6
7	Tolerância a falhas	6
7.1	Heartbeat centralizado	6
7.2	Deteção da queda do Coordenador	6
7.3	Eleição do Valentão (<i>Bully</i>)	6
7.4	Recuperação	7
7.5	Modelo de falha do host do Coordenador	7
8	Cenários de teste obrigatórios	7
9	Execução	8

1 Introdução

O **Shared Distributed Write Board** (SDWB) é um quadro branco colaborativo no qual múltiplos terminais desenham em tempo real sobre uma mesma tela compartilhada. O sistema *não* possui um servidor fixo: a descoberta dos quadros é feita por um **Serviço de Nomes** e a ordenação das operações fica a cargo de um **Coordenador migrante** — um dos próprios nós que, ao falhar, é substituído por outro por meio de uma eleição. Esse arranjo exige lidar com os três problemas centrais de qualquer sistema distribuído real: **descoberta**, **consenso** (ordenação e exclusão mútua) e **recuperação** (tolerância a falhas).

A arquitetura adotada é a de uma **máquina de estado replicada**: todo nó mantém uma réplica completa do quadro (objetos, lista de participantes e tabela de travas). O Coordenador não é dono do estado; ele apenas *ordena* as operações — atribuindo um número de sequência — e as *difunde* (*broadcast*) a todos. Como o estado já está replicado em todos os nós, quando o Coordenador cai o vencedor da eleição assume sem reconstruir nada.

A comunicação usa **gRPC** [gRP24] sobre Python 3, com o arquivo `sdwb.proto` servindo de contrato único e documentação do protocolo. A interface gráfica é construída em **Tkinter**. Toda a lógica distribuída — exclusão mútua, detecção de falhas e eleição — foi implementada manualmente, sem coordenadores externos prontos (Zookeeper, etcd, Consul).

Decisão de escopo. O enunciado oficial apresenta a seção de *Controle de Transações Distribuídas (2PC)* **riscada** (tachada). Por orientação do professor, o protocolo *Two-Phase Commit* foi mantido fora do escopo: não há **Prepare/Commit/Abort** nem operações atômicas em grupo. A coordenação de concorrência deste trabalho se dá exclusivamente pela **exclusão mútua** por objeto (Seção 6).

2 Arquitetura

O SDWB é uma **máquina de estado replicada com coordenador migrante**. Não é um sistema cliente-servidor clássico: o estado completo do quadro vive replicado em todos os nós, e o papel de Coordenador é apenas um *serviço* que pode rodar em qualquer nó e migrar entre eles. O sistema tem três componentes.

- **Serviço de Nomes** — as “Páginas Amarelas”. Processo separado, com endereço fixo, que guarda *somente* a tabela (`nome_do_quadro`, `ip`, `porta`) dos Coordenadores. Por enunciado, nunca falha; logo, é mantido simples, sem replicação.
- **Coordenador do Quadro** — serviço que *ordena* as operações (atribui número de sequência e identificadores), faz *broadcast* para todos, mantém a lista de membros, a tabela de travas e conduz a eleição. “Coordenador = quadro”: é um serviço, não uma máquina.
- **Cliente / Nó** — interface gráfica (Tkinter) mais a réplica local do estado. Cada nó sobe um **servidor gRPC sempre ativo**, mesmo quando é apenas cliente — é o que lhe permite receber mensagens de eleição e, eventualmente, tornar-se Coordenador.

2.1 Coordenador in-process e migração

A decisão arquitetural central é que **cada nó hospeda, no mesmo processo, um único servidor gRPC** que publica dois serviços: No (eleição, sempre ativo) e Coordenador (ativo apenas no nó que coordena o quadro; nos demais fica dormiente e rejeita chamadas). Isso reflete diretamente o enunciado — “o Coordenador é um dos próprios nós” e “quando o cliente onde está o Coordenador sai do quadro, este mesmo cliente continua hospedando o quadro”. Quando um nó cria um quadro ou vence uma eleição, ele simplesmente *ativa* o seu Coordenador com a réplica que já mantinha; não há transferência de estado nem processos auxiliares.

2.2 Modelo de dados

Cada elemento desenhável é um Objeto:

```
1 Objeto = { id, tipo: LINHA|QUADRADO, pontos: [[x,y],...], cor, dono }
```

Listing 1: Modelo de objeto (sdwb.proto)

O `id` é atribuído pelo Coordenador (garante unicidade global). O estado do quadro é o conjunto: dicionário de objetos + lista de membros (`id_cliente`, `ip`, `porta`) + tabela de travas `{objeto_id: id_cliente}` + último número de sequência.

2.3 Organização do código

O projeto separa rede de interface: `backend/` contém o contrato e a lógica distribuída — `sdwb.proto`, os *stubs* gerados, `servico_nomes.py`, `coordenador.py` (estado do quadro + he-

artbeat) e `no.py` (o Nó: servidor gRPC, réplica, heartbeat e eleição). O `frontend/cliente.py` é uma camada de visualização fina sobre um No.

3 Protocolo de mensagens (gRPC)

O arquivo `sdwb.proto` é a fonte única do protocolo. Os *stubs* são gerados com:

```
1 python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=.
   sdwb.proto
```

3.1 Serviço ServicoNomes

RPC	Entrada	Quando
RegistrarQuadro	InfoQuadro	Coordenador cria um quadro
ListarQuadros	Vazio	Cliente busca quadros disponíveis
AtualizarQuadro	InfoQuadro	Novo Coordenador (pós-eleição)
RemoverQuadro	NomeQuadro	Quadro encerrado

Tabela 1: RPCs do Serviço de Nomes. InfoQuadro = (nome, ip, porta).

3.2 Serviço Coordenador

Hospedado pelo nó que coordena o quadro.

RPC	Tipo	Função
Join	<i>server-streaming</i>	Onboarding + broadcast
EnviarOperacao	unário	Desenhar/colorir/remover
Selecionar	unário	Adquirir trava de um objeto
Liberar	unário	Liberar a trava
Heartbeat	unário	Sinal de vida periódico
Sair	unário	Saída graciosa

Tabela 2: RPCs do Coordenador.

O Join é o coração do sistema. O cliente o chama *uma vez*; recebe primeiro um evento ESTADO_INICIAL (*snapshot* completo) e mantém a conexão aberta, pela qual o Coordenador *empurra* cada evento novo:

```
1 rpc Join(PedidoJoin) returns (stream Evento);
```

Listing 2: Streaming do Join

Os eventos difundidos são tipados por `TipoEvento`:

TipoEvento	Carga
ESTADO_INICIAL	<i>snapshot</i> completo (objetos, membros, travas, seq)
OPERACAO	operação ordenada (OperacaoAplicada: seq + objeto)
MEMBROS	lista de membros atualizada
LOCK	tabela de travas atualizada
NOVO_COORDENADOR	endereço do novo coordenador (pós-eleição)

Tabela 3: Eventos empurrados pela stream do Join.

Uma `Operacao` carrega `TipoOperacao` $\in \{\text{CRIAR, COLORIR, REMOVER}\}$, o `id_cliente` autor e os campos pertinentes (objeto a criar, ou `objeto_id` e `cor`).

3.3 Serviço No (eleição)

Hospedado por *todos* os nós, sempre ativo.

RPC	Entrada	Função
Eleicao	PedidoEleicao	Valentão: aviso de eleição a IDs maiores
AnunciarCoordenador	AnuncioCoordenador	Vencedor divulga o novo coordenador

Tabela 4: RPCs de eleição.

4 Descoberta e *onboarding*

O endereço do Coordenador nunca é fixo no cliente. O fluxo de entrada é:

1. o cliente consulta `ListarQuadros` no Serviço de Nomes e escolhe um quadro;
2. conecta-se ao Coordenador daquele quadro e chama `Join`, passando seu (`id_cliente`, `ip`, `porta`);
3. recebe `ESTADO_INICIAL` com todos os desenhos e a lista de participantes.

Critério de aceite atendido: um novo nó vê todos os desenhos existentes imediatamente ao entrar.

5 Ordenação e consistência

Toda operação passa pelo Coordenador, que lhe atribui um número de sequência crescente e a difunde a todos os membros — *inclusive ao autor*. A interface **não renderiza localmente** antes do broadcast: o desenho só aparece quando o evento `OPERACAO` retorna pela stream. Como todos aplicam as operações na mesma ordem (a ordem de sequência), as réplicas convergem identicamente. A criação de objetos também é serializada no Coordenador, que gera ids globais (`obj-1`, `obj-2`, ...), evitando colisões.

6 Exclusão mútua

Antes de *colorir* ou *remover*, o cliente precisa **selecionar** o objeto. O Coordenador mantém a tabela {objeto_id: id_cliente}; ao receber **Selecionar**, concede a trava se o objeto estiver livre e **recusa** (mensagem de erro) se já estiver travado por outro cliente. A operação de **COLORIR/REMOVER** é validada no Coordenador: só prossegue se o autor detém a trava, que é liberada automaticamente após a operação. Travar/liberar gera um evento **LOCK** difundido a todos, permitindo destacar visualmente o objeto em uso (traço grosso para o próprio, tracejado para travas de outros).

Por que centralizada. Há três algoritmos clássicos de exclusão mútua distribuída [TvS17]: *centralizado* (um coordenador concede/nega o acesso), *token ring* (um *token* circula em anel) e *distribuído* (Ricart–Agrawala, pedindo permissão a todos). O SDWB usa o **algoritmo centralizado**: ele é o coerente com a arquitetura, que já possui um Coordenador ordenando todas as operações. *Token ring* e Ricart–Agrawala seriam apropriados na ausência de um coordenador; aqui seriam redundantes. O ponto único de falha do Coordenador é tratado não por outro algoritmo de exclusão, mas pela **eleição** (Seção 7): o novo Coordenador assume já com a tabela de travas replicada.

Critério de aceite atendido: dois clientes não conseguem operar o mesmo objeto ao mesmo tempo.

7 Tolerância a falhas

7.1 Heartbeat centralizado

Cada cliente envia **Heartbeat** ao Coordenador a cada T segundos ($T = 2s$). O Coordenador registra o instante do último sinal de cada membro; uma thread monitora a tabela e **remove** quem passar de $2T$ sem dar notícia, difundindo a lista de membros atualizada. A ausência de notificação é a forma centralizada de detecção de falha exigida pelo enunciado.

7.2 Detecção da queda do Coordenador

Para o cliente, a falha da própria conexão com o Coordenador é o gatilho da eleição. Isso é detectado de duas maneiras complementares: (i) a *stream* do **Join** é encerrada inesperadamente, ou (ii) uma chamada de **Heartbeat** falha. Qualquer das duas dispara o procedimento de eleição (protegido por uma trava para não disparar duas vezes).

7.3 Eleição do Valentão (*Bully*)

Cada nó possui um **ID** inteiro, derivado de **ip:porta**. Ao detectar a queda do Coordenador, o nó executa o algoritmo do Valentão [GM82]:

1. envia **Eleicao** a todos os membros de **ID maior** que o seu (exceto o Coordenador morto);

2. se *algum* responder, recua e aguarda o anúncio do novo Coordenador (com *timeout* e nova tentativa se nada chegar);
3. se *nenhum* responder, declara-se vencedor.

Quem recebe um `Eleicao` de um ID menor responde “vivo” e inicia a sua própria eleição — garantindo que o nó de maior ID entre os sobreviventes prevaleça.

7.4 Recuperação

O vencedor reconstrói o estado a partir da sua própria réplica (objetos, membros, travas e sequência) — método `EstadoCoordenador.de_snapshot` — ativa o seu serviço `Coordenador`, chama `AtualizarQuadro` no Serviço de Nomes com o novo endereço e envia `AnunciarCoordenador` aos demais. Cada nó, ao receber o anúncio, reconecta o `Join` ao novo Coordenador e recebe um novo `ESTADO_INICIAL`. Como o estado já estava replicado, **nada é perdido**.

7.5 Modelo de falha do host do Coordenador

O enunciado distingue “sair” (saída graciosa, com mensagem) de “cair” (queda detectada por *timeout*); são caminhos de código distintos:

- **Sai** (botão Sair) e **há outros membros** $\Rightarrow$ o Coordenador *continua* naquela mesma máquina; **sem eleição**.
- **Cai** (processo morto / janela fechada) e **há outros membros** $\Rightarrow$ eleição $\rightarrow$ novo host assume $\rightarrow$ atualiza o Serviço de Nomes.
- **Coordenador sozinho** e sai ou cai $\Rightarrow$ o quadro **encerra** (removido do Serviço de Nomes).

Critério de aceite atendido: o sistema continua operando após a queda de qualquer nó.

8 Cenários de teste obrigatórios

Os três cenários do enunciado são cobertos por testes de fumaça automatizados, executáveis em conjunto por `python rodar_testes.py`.

Cenário	Teste	Verifica
1. Entrada dinâmica	<code>teste_fumaca_quadro.py</code>	descoberta, onboarding, ordenação, broadcast
2. Concorrência	<code>teste_fumaca_exclusao.py</code>	exclusão mútua por objeto
3. Morte do coord.	<code>teste_fumaca_eleicao.py</code>	eleição do Valentão + migração

Tabela 5: Mapeamento cenário $\rightarrow$ teste. Todos passam (5/5 com os auxiliares).

No cenário 3, três nós entram num quadro; o Coordenador (que o criou) é morto; os sobreviventes detectam a queda, elegem o de maior ID, que atualiza o Serviço de Nomes e mantém

o objeto desenhado antes da falha — e o sistema segue aceitando novas operações.

9 Execução

1. Serviço de Nomes (porta 50000): `cd backend && python servico_nomes.py`.
2. Clientes: `cd frontend && python cliente.py` (uma janela por participante). *Criar novo quadro* torna o nó coordenador; *Ingressar* conecta a um quadro existente.
3. Demonstrar falha: matar o processo do nó coordenador — os demais elegem um novo automaticamente.

Referências

- [CDKB11] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. *Distributed Systems: Concepts and Design*. Addison-Wesley, 5 edition, 2011.
- [GM82] Hector Garcia-Molina. Elections in a distributed computing system. *IEEE Transactions on Computers*, C-31(1):48–59, 1982.
- [Goo24] Google. Protocol Buffers: Language Guide (proto3). <https://protobuf.dev/programming-guides/proto3/>, 2024. Acesso em 2026.
- [gRP24] gRPC Authors. gRPC: A high performance, open source universal RPC framework. <https://grpc.io/docs/>, 2024. Acesso em 2026.
- [TvS17] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms*. Pearson, 3 edition, 2017.